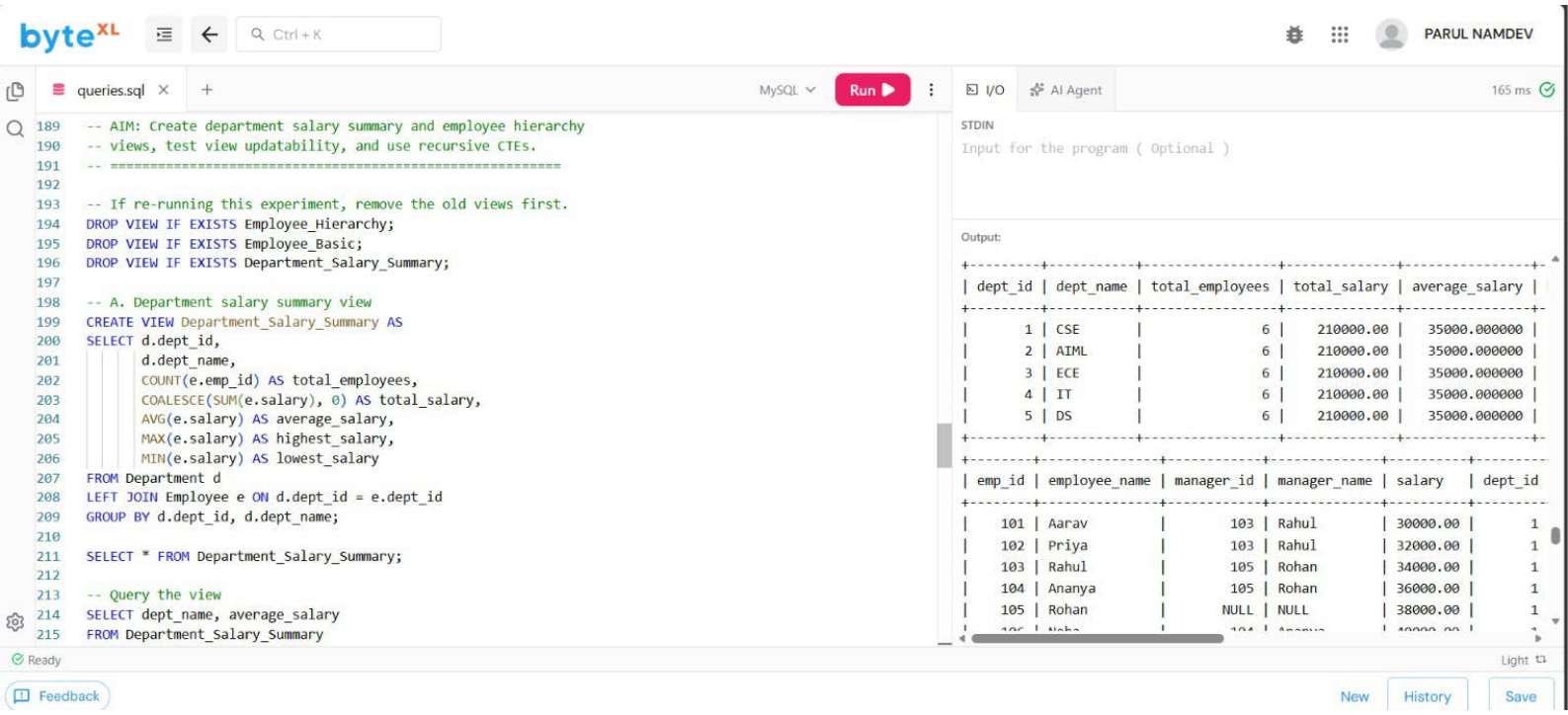
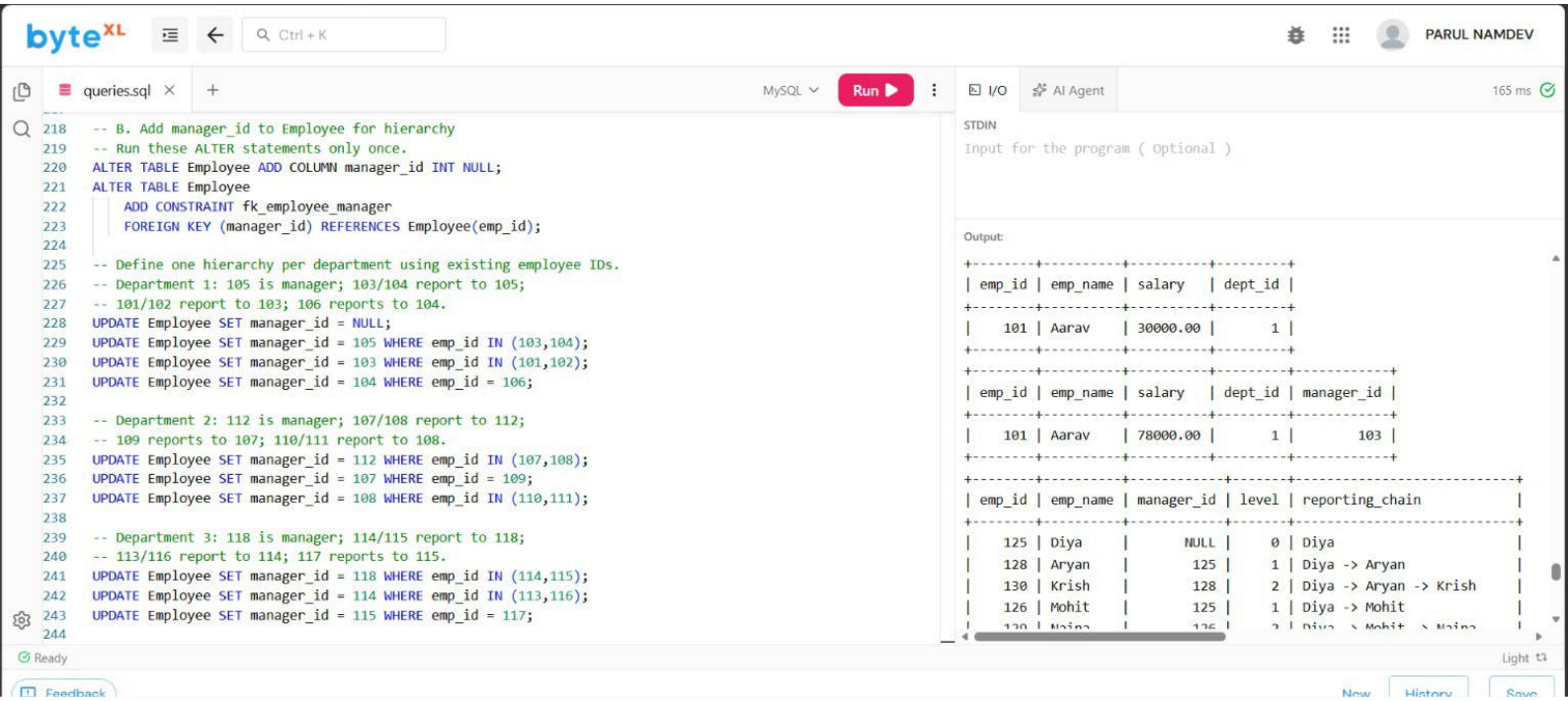
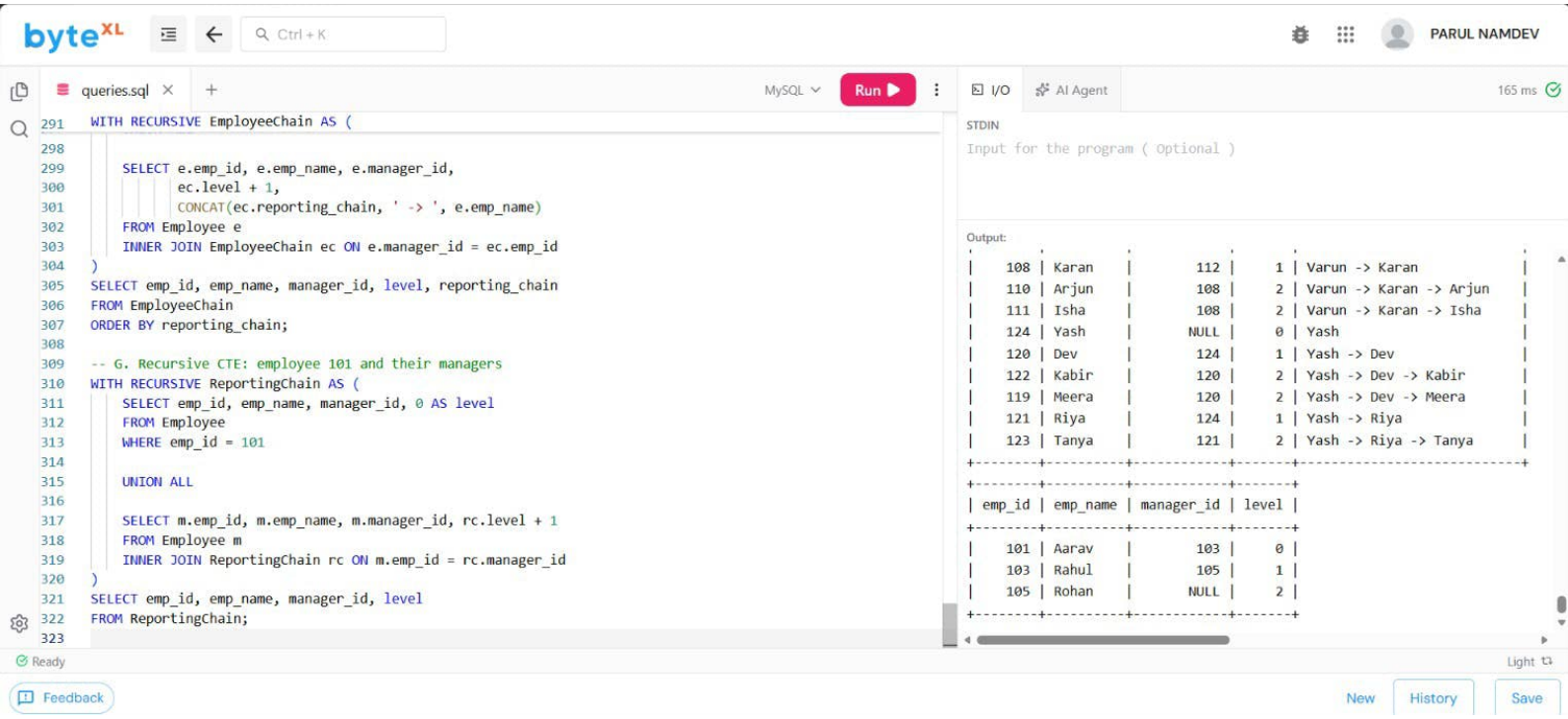

byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
165 ms
STDIN
Input for the program (Optional)
Output:
-- AIM: Create department salary summary and employee hierarchy
-- views, test view updatability, and use recursive CTEs.
-- ==
-- If re-running this experiment, remove the old views first.
DROP VIEW IF EXISTS Employee_Hierarchy;
DROP VIEW IF EXISTS Employee_Basic;
DROP VIEW IF EXISTS Department_Salary_Summary;
-- A. Department salary summary view
CREATE VIEW Department_Salary_Summary AS
SELECT d.dept_id,
d.dept_name,
COUNT(e.emp_id) AS total_employees,
COALESCE(SUM(e.salary), 0) AS total_salary,
AVG(e.salary) AS average_salary,
MAX(e.salary) AS highest_salary,
MIN(e.salary) AS lowest_salary
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
SELECT * FROM Department_Salary_Summary;
-- Query the view
SELECT dept_name, average_salary
FROM Department_Salary_Summary
dept_id	dept_name	total_employees	total_salary	average_salary
1	CSE	6	210000.00	35000.000000
2	AIML	6	210000.00	35000.000000
3	ECE	6	210000.00	35000.000000
4	IT	6	210000.00	35000.000000
5	DS	6	210000.00	35000.000000
emp_id	employee_name	manager_id	manager_name	salary	dept_id
101	Aarav	103	Rahul	30000.00	1
102	Priya	103	Rahul	32000.00	1
103	Rahul	105	Rohan	34000.00	1
104	Ananya	105	Rohan	36000.00	1
105	Rohan	NULL	NULL	38000.00	1
Ready
Light
Feedback
New
History
Save


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
165 ms
STDIN
Input for the program (Optional)
Output:
-- B. Add manager_id to Employee for hierarchy
-- Run these ALTER statements only once.
ALTER TABLE Employee ADD COLUMN manager_id INT NULL;
ALTER TABLE Employee
ADD CONSTRAINT fk_employee_manager
FOREIGN KEY (manager_id) REFERENCES Employee(emp_id);
-- Define one hierarchy per department using existing employee IDs.
-- Department 1: 105 is manager; 103/104 report to 105;
-- 101/102 report to 103; 106 reports to 104.
UPDATE Employee SET manager_id = NULL;
UPDATE Employee SET manager_id = 105 WHERE emp_id IN (103,104);
UPDATE Employee SET manager_id = 103 WHERE emp_id IN (101,102);
UPDATE Employee SET manager_id = 104 WHERE emp_id = 106;
-- Department 2: 112 is manager; 107/108 report to 112;
-- 109 reports to 107; 110/111 report to 108.
UPDATE Employee SET manager_id = 112 WHERE emp_id IN (107,108);
UPDATE Employee SET manager_id = 107 WHERE emp_id = 109;
UPDATE Employee SET manager_id = 108 WHERE emp_id IN (110,111);
-- Department 3: 118 is manager; 114/115 report to 118;
-- 113/116 report to 114; 117 reports to 115.
UPDATE Employee SET manager_id = 118 WHERE emp_id IN (114,115);
UPDATE Employee SET manager_id = 114 WHERE emp_id IN (113,116);
UPDATE Employee SET manager_id = 115 WHERE emp_id = 117;
+--------+----------+----------+---------+
| emp_id | emp_name | salary | dept_id |
+--------+----------+----------+---------+
| 101 | Aarav | 30000.00 | 1 |
+--------+----------+----------+---------+
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary | dept_id | manager_id |
+--------+----------+----------+---------+------------+
| 101 | Aarav | 78000.00 | 1 | 103 |
+--------+----------+----------+---------+------------+
+--------+----------+------------+-------+----------------------------+
| emp_id | emp_name | manager_id | level | reporting_chain |
+--------+----------+------------+-------+----------------------------+
125	Diya	NULL	0	Diya
128	Aryan	125	1	Diya -> Aryan
130	Krish	128	2	Diya -> Aryan -> Krish
126	Mohit	125	1	Diya -> Mohit
Ready
Light
Feedback
New
History
Save




```
WITH RECURSIVE EmployeeChain AS (

    SELECT e.emp_id, e.emp_name, e.manager_id,
           ec.level + 1,
           CONCAT(ec.reporting_chain, ' -> ', e.emp_name)
    FROM Employee e
    INNER JOIN EmployeeChain ec ON e.manager_id = ec.emp_id
)
SELECT emp_id, emp_name, manager_id, level, reporting_chain
FROM EmployeeChain
ORDER BY reporting_chain;

-- G. Recursive CTE: employee 101 and their managers
WITH RECURSIVE ReportingChain AS (
    SELECT emp_id, emp_name, manager_id, 0 AS level
    FROM Employee
    WHERE emp_id = 101

    UNION ALL

    SELECT m.emp_id, m.emp_name, m.manager_id, rc.level + 1
    FROM Employee m
    INNER JOIN ReportingChain rc ON m.emp_id = rc.manager_id
)
SELECT emp_id, emp_name, manager_id, level
FROM ReportingChain;
```

```
+--------+----------+------------+-------+----------------------------+
|    108 | Karan    |        112 |     1 | Varun -> Karan             |
|    110 | Arjun    |        108 |     2 | Varun -> Karan -> Arjun    |
|    111 | Isha     |        108 |     2 | Varun -> Karan -> Isha     |
|    124 | Yash     |       NULL |     0 | Yash                       |
|    120 | Dev      |        124 |     1 | Yash -> Dev                |
|    122 | Kabir    |        120 |     2 | Yash -> Dev -> Kabir       |
|    119 | Meera    |        120 |     2 | Yash -> Dev -> Meera       |
|    121 | Riya     |        124 |     1 | Yash -> Riya               |
|    123 | Tanya    |        121 |     2 | Yash -> Riya -> Tanya      |
+--------+----------+------------+-------+----------------------------+
+--------+----------+------------+-------+
| emp_id | emp_name | manager_id | level |
+--------+----------+------------+-------+
|    101 | Aarav    |        103 |     0 |
|    103 | Rahul    |        105 |     1 |
|    105 | Rohan    |       NULL |     2 |
+--------+----------+------------+-------+
```